

Objectifs

À l'issue de cette section, l'étudiant devra être capable de :
 — ok

1 Algorithme glouton

Contrairement au retour sur trace, un *algorithme glouton* adopte une stratégie qui ne revient jamais en arrière. Il construit une solution à un problème en se fondant sur des considérations *locales*. Cette stratégie, souvent appliquée pour résoudre des problèmes d'optimisation, fournit généralement une solution qui est optimale mais pas nécessairement meilleure.

On présente une mise en œuvre de cette approche pour résoudre des problèmes.

1.1 Coloration d'un graphe

Coloration d'un graphe.

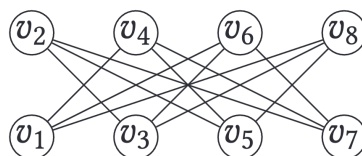
Étant donné un graphe non orienté et non pondéré $G = (V, E)$, on appelle k -coloration de G , pour $k \in \mathbb{N}^*$, une application c qui associe à chacun des sommets de G un entier de $[0, k - 1]$ de sorte que si deux sommets u et v sont voisins alors leurs couleurs sont différentes :

$$\forall (u, v) \in E, \quad c(u) \neq c(v).$$

Le problème de la coloration d'un graphe consiste à trouver une k -coloration pour laquelle la valeur de k est minimale.

Considérons le graphe ci-dessous et un premier ordre de parcours des sommets :

$v_1, v_2, v_3, v_4, v_5, v_6, v_7, v_8.$



Un algorithme glouton affecte une première couleur au sommet v_1 : $c(v_1) = 0$.

Puis le sommet v_2 est visité. N'étant pas adjacent au sommet v_1 , il peut être colorié avec la même couleur : $c(v_2) = 0$.

Vient la coloration de v_3 , sommet adjacent à v_2 . Il ne peut pas être colorié avec la couleur 0. L'algorithme lui attribue la plus petite couleur différente de celle de ses voisins : $c(v_3) = 1$.

Une même analyse mène à la coloration de v_4 avec $c(v_4) = 1$.

Le sommet v_5 est adjacent à v_2 et v_4 , sommets déjà coloriés. Alors $c(v_5) = 2$.

Et ainsi de suite avec tous les autres sommets.

Au final, la coloration obtenue en suivant l'ordre choisi est :

$$c(v_1) = 0 \quad c(v_3) = 1 \quad c(v_5) = 2 \quad c(v_7) = 3$$

$$c(v_2) = 0 \quad c(v_4) = 1 \quad c(v_6) = 2 \quad c(v_8) = 3$$

Il s'agit d'une 4-coloration.

Pourtant, une rapide observation du graphe montre que ce dernier est biparti. L'ensemble V de ses sommets peut être partitionné en deux sous-ensembles disjoints $V_1 = \{v_1, v_3, v_5, v_7\}$ et $V_2 = \{v_2, v_4, v_6, v_8\}$. En conséquence, une 2-coloration est possible sous la forme :

$$c(v_1) = 0 \quad c(v_3) = 0 \quad c(v_5) = 0 \quad c(v_7) = 0$$

$$c(v_2) = 1 \quad c(v_4) = 1 \quad c(v_6) = 1 \quad c(v_8) = 1$$

Ces deux situations illustrent bien le fait que l'algorithme peut trouver une solution, comme par exemple la 4-coloration, mais pas nécessairement la meilleure, comme la 2-coloration. Il ne trouve pas la pire solution, à savoir une 8-coloration, où chaque sommet se verrait attribuer une couleur différente de celle de tous les autres sommets.

De manière générale, le nombre de sommets constitue trivialement une borne supérieure pour la coloration d'un graphe. Le programme n'utilise jamais plus de $1 + \Delta$ couleurs où Δ est le degré maximal du graphe supposé connexe.

Théorème de Brooks

- Un graphe complet admet $1 + \Delta$ couleurs exactement sont nécessaires.
- Un graphe réduit à un cycle de longueur impaire admet $1 + \Delta$ couleurs exactement.

Quel ordre ?

La détermination des couleurs dépend étroitement de l'ordre dans lequel les sommets d'un graphe sont visités. On peut alors s'interroger sur l'existence d'un ordre particulier qui fournirait la coloration optimale. Malheureusement, ce problème est NP-difficile en tant que sous-problème de la coloration optimale d'un graphe, lui-même problème NP-complet.

Le programme suivante définit une fonction `greedy_color` qui applique la stratégie gloutonne et qui renvoie un tableau des couleurs attribuées à chaque sommet.

Algorithme glouton de coloration de graphe

```

1  (* la plus petite couleur qui n'est pas dans l *)
2  let min_color (l: int list) : int =
3    let n = List.length l in
4    let free = Array.make (n + 1) true in
5    List.iter (fun v -> free.(v) <- false) l;
6    let rec find i = if free.(i) then i else find (i + 1) in
7    find 0
8

```

```

9 let greedy_color (g: graph) (order: int list) : int array =
10   let color = Array.make (size g) 0 in
11   let assign v =
12     let colors = List.map (fun w -> color.(w)) (succ g v) in
13     color.(v) <- min_color colors in
14   List.iter assign order;
15   color

```

Cette fonction reçoit deux arguments : un graphe g et la liste `order` précisant l'ordre de visite des sommets. Le code est décomposé en deux parties.

- Une fonction `min_color` qui détermine la plus petite couleur n'apparaissant pas dans une liste.
- La fonction `greedy_color` qui parcourt la liste `order` et affecte une couleur à chaque sommet avec la fonction `assign`. Celle-ci calcule la liste des couleurs utilisées par les voisins, avant d'appeler `min_color`.

1.2 Rendu de monnaie.

Considérons un ensemble de n billets et pièces de monnaie de valeurs entières $v_1 < v_2 < \dots < v_n$, avec $v_1 = 1$ (pour nous 1€, 2€, 5€, 10€, 20€, 50€, 100€, 200€, 500€).

Le problème du *rendu de monnaie* consiste à déterminer un nombre minimal de billets et de pièces pour rendre une somme donnée.

Exemple

La somme de 49€ peut être rendue avec 49 pièces de 1€ mais aussi avec 2 billets de 20€, 1 billet de 5€ et 2 pièces de 2€, ce qui représente un nombre de 5 billets et pièces rendus au lieu des 49 pièces. Ce nombre 5 est d'ailleurs le plus petit nombre de billets et de pièces rendus.

Il s'agit là encore d'un problème d'optimisation.

Même si elle ne fournit pas toujours la meilleure solution possible, une *stratégie gloutonne* est souvent adoptée en raison de sa simplicité de mise en œuvre

- On choisit d'abord les billets qui permettent de rendre la plus grande valeur possible sur la somme à rendre. Dans notre exemple, on choisit d'abord 2 billets de 20€.
- Il reste alors à rendre 9€. On choisit 1 billet de 5€, la plus grande valeur inférieure ou égale à 9€ qui peut être rendue.
- Il reste enfin 4€, qui peuvent être rendus avec 2 pièces de 2€.

Le programme suivante définit une fonction `greedy_change` qui renvoie un tableau des nombres de billets et de pièces utilisés pour rendre une somme v , étant donné un système monétaire ordonné `coins`.

Algorithme glouton de rendu de monnaie

```

1 let greedy_change (coins: int array) (v: int) : int array =
2   let n = Array.length coins in
3   let change = Array.make n 0 in

```

```

4  let cur = ref v in
5  let i = ref (n - 1) in
6  while !cur > 0 do
7    let c = coins.(!i) in
8    if !cur < c then
9      decr i
10   else (
11     change.(!i) <- change.(!i) + 1;
12     cur := !cur - c
13   )
14 done;
15 change

```

On note que la correction du programme est assurée par l'hypothèse $v_1 = 1$. En particulier, cela garantit l'invariant de boucle $i \geq 0$ et un accès valide au tableau `coins`.

On peut démontrer que cette stratégie gloutonne donne systématiquement un rendu de monnaie optimal avec le système monétaire de la zone euro. À ce titre, ce dernier est qualifié de *canonique*.

Théorème - Optimalité du rendu de monnaie glouton en euros

Si `coins` décrit le système monétaire de la zone euro, alors le programme calcule un rendu de monnaie utilisant un nombre minimal d'éléments.

Démonstration. Dans la preuve, on utilise le mot « pièce » pour désigner indistinctement les pièces et les billets.

Commençons par observer que certaines combinaisons de pièces ne peuvent pas se trouver dans une solution optimale.

- Une pièce de valeur 1, 5, 10, 50 ou 100 n'est jamais utilisée deux fois dans une solution optimale. En effet, pour chacune il existe une pièce de valeur double remplaçant avantageusement deux occurrences de la première.
- Une pièce de valeur 2 ou 20 n'est jamais utilisée trois fois dans une solution optimale. En effet, trois pièces de valeur 2 sont avantageusement remplaçables par une pièce de valeur 1 et une de valeur 5, et similairement pour 20.
- Une pièce de valeur 1 (resp. 10) n'accompagne jamais deux pièces de valeur 2 (resp. 20) dans une solution optimale. En effet, on pourrait les remplacer avantageusement par une unique pièce de valeur 5 (resp. 50).

En combinant toutes ces contraintes on détermine que, dans un rendu de monnaie optimal, les pièces de valeur inférieure ou égale à 100 ne peuvent pas compter pour plus de $2 \times 2 + 5 + 2 \times 20 + 50 + 100 = 199$.

Si la somme à rendre est supérieure ou égale à 200, la solution optimale contient donc nécessairement un billet de 200, que notre algorithme glouton a raison de choisir.

On reproduit ensuite cette dernière étape du raisonnement pour chaque valeur de nos pièces : les pièces de valeur inférieure ou égale à 50 ne peuvent pas compter pour plus de 99, celles de valeur inférieure ou égale à 20 ne peuvent pas compter pour plus de 49, etc.

Le programme ne sélectionne donc que des pièces qui font nécessairement partie de la solution optimale.

2 Décomposition d'un problème en sous-problèmes

La résolution d'un problème peut parfois être menée en le décomposant en sous-problèmes. Une fois ces derniers résolus, leurs solutions sont combinées pour construire la solution du problème initial.

On distingue deux types de problèmes selon que les sous-problèmes sont indépendants ou dépendants menant aux stratégies dites *diviser pour régner* ou de *programmation dynamique*.

2.1 Diviser pour régner

De nombreuses solutions algorithmiques à certaines instances de problèmes sont obtenues en résolvant des instances de sous-problèmes indépendants. Chaque sous-problème est à son tour découpé en sous-problèmes jusqu'à aboutir à des situations simples, appelées *cas de base*. La solution de ces derniers est généralement simple à obtenir, voire est une donnée du problème, et permet alors la construction de la solution globale du problème initial.

Cette approche de résolution porte le nom de *diviser pour régner*.

En pratique, l'instance d'un problème peut souvent être caractérisée par sa taille n . Adopter le paradigme diviser pour régner, c'est donc résoudre une instance d'un problème de taille n en résolvant le même problème sur des instances de tailles strictement inférieures à n : exponentiation rapide, recherche dichotomique dans un tableau trié, tri rapide d'un tableau, tri fusion récursif de tableau.

Exemple

Le calcul de a^n pour des entiers a et n avec $n \geq 0$ a le principe suivante :

$$\begin{cases} \text{power } a \ 0 = 1 & \text{si } n = 0 \\ \text{power } a \ (2k) = (\text{power } a \ k)^2 & \text{si } n = 2k \text{ avec } k > 0 \\ \text{power } a \ (2k + 1) = a \times (\text{power } a \ k)^2 & \text{si } n = 2k + 1 \text{ avec } k \geq 0 \end{cases}$$

La première ligne de ce calcul correspond à un cas de base pour lequel $n = 0$ et $a^n = 1$. Pour toute autre valeur non nulle de n , les deux lignes suivantes précisent les modalités de calcul, selon la parité de n . Dans les deux cas, le calcul se poursuit avec une valeur moitié $\lfloor n/2 \rfloor$ du second argument de la fonction par rapport à la valeur initiale n .

De fait, le calcul $\text{power } a \ n$ est décomposé en celui de $\text{power } a \ \lfloor n/2 \rfloor$, ce dernier étant lui-même décomposé en celui de $\text{power } a \ \lfloor n/2 \rfloor$ et ainsi de suite jusqu'à atteindre le cas de base.

Chaque calcul intermédiaire se fait sur une instance de taille strictement plus petite, ici la moitié arrondie inférieurement, que celle du calcul initial.

Mais ce n'est pas le seul trait qui caractérise l'approche diviser pour régner : les sous-problèmes menant à la résolution du problème général doivent être indépendants !

D'une certaine manière, tous les problèmes s'emboîtent, le gros problème dépendant des problèmes moyens, eux-mêmes dépendant des petits problèmes, eux-mêmes dépendant enfin des cas de base sans qu'il y ait de redondances dans les calculs.

Cette propriété fondamentale autorise la construction directe de la solution globale par recombinaison des solutions intermédiaires.

2.2 Schéma de résolution.

On peut dégager une procédure de construction d'une solution générale par l'approche diviser pour régner en trois étapes essentielles.

- **Diviser.** Cette étape décompose le problème initial en un ou plusieurs sous-problèmes de plus petites tailles.
- **Régner.** Chaque sous-problème est résolu.
- **Rassembler.** Les solutions des sous-problèmes sont rassemblées pour construire la solution du problème initial.

La division répétée des problèmes mène inévitablement à des problèmes élémentaires qui constituent des *cas de base* pour lesquels la solution est soit immédiate, soit obtenue par un traitement élémentaire. De fait, cette procédure se prête naturellement à un traitement récursif des problèmes mais une approche impérative est tout autant adaptée.

Une conséquence immédiate de cette procédure est l'assurance de la *correction* de l'algorithme.

Seule reste à valider sa *terminaison*, à savoir que, pour un problème initial de taille quelconque, on atteint toujours le cas de base.

Exemple

Schéma de résolution du tri fusion

```
void mergesortrec(int a[], int tmp[], int l, int r) {
    if (r - l <= 1) return;
    int m = l + (r - l) / 2;
    mergesortrec(a, tmp, l, m);
    mergesortrec(a, tmp, m, r);
    if (a[m-1] <= a[m]) return; // optimisation
    for (int i = l; i < r; i++) tmp[i] = a[i];
    merge(tmp, a, l, m, r);
}
```

Le tri d'un tableau d'entiers entre les indices l et r (exclu) est décomposé en deux sous-tris par les appels récursifs à la fonction `mergesortrec`.

- Le premier appel trie la première moitié du tableau, d'indices compris entre l et m , indice milieu du tableau.
- Le second appel trie la seconde moitié du tableau, d'indices compris entre m et r .

Ceci correspond d'une part à l'étape *diviser*, puisque deux sous-tableaux du tableau initial sont traités, mais également à l'étape *régner* puisque la fonction trie les deux moitiés de a .

L'appel à la fonction `merge` rassemble alors les sous-tableaux en les fusionnant, de manière ordonnée, dans a .

Dans ce programme, le *cas de base* correspond au test fait sur la première ligne de la fonction : si le tableau à trier est de taille 0 ou 1, il est déjà trié !

Exemple

La recherche dichotomique d'un élément v dans un tableau a a comme étape *diviser* la recherche de v dans l'un des sous-tableaux de a : soit v est dans la moitié gauche de a , soit v est dans sa moitié droite.

L'étape *régner* trouve v dans le sous-tableau précédemment identifié.

Ces deux étapes sont répétées jusqu'à aboutir à un cas de base : soit v est trouvée, soit elle ne l'est pas. Ici, il n'y a rien à rassembler, cette étape se réduisant simplement à renvoyer le résultat du cas de base.

Les exemples précédents ont en commun des divisions de tableaux par moitié. Mais d'autres divisions sont possibles, en un nombre plus important de parties. De même, la division peut mener à des sous-problèmes dans lesquels les données initiales ne sont plus contiguës mais *entrelacées*.

C'est par exemple le cas dans le tri fusion d'une liste en OCaml où la fonction `split` divise les éléments d'une liste en deux sous-listes d'éléments entrelacés.

2.3 Complexité.

Les complexités temporelles des programmes adoptant la stratégie *diviser pour régner* prennent de nombreuses formes selon la nature des divisions et des recombinaisons.

Quelques relations types reviennent cependant assez régulièrement.

La relation suivante est fréquente quand le problème est divisé en deux sous-problèmes de tailles moitiés :

$$C(n) = C\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + C\left(\left\lceil \frac{n}{2} \right\rceil\right) + f(n)$$

Les deux premiers termes du membre de droite correspondent au coût de la division et des traitements afférents. Le dernier terme correspond au coût des traitements pré-division et de recombinaison (par exemple, pour le tri fusion, $f(n) = 6n$ et on trouve alors $C(n) = O(n \log n)$).

2.4 Deux plus proches points du plan

Considérons un ensemble de $n \geq 3$ points du plan définis par leurs coordonnées cartésiennes dans un repère orthonormé.

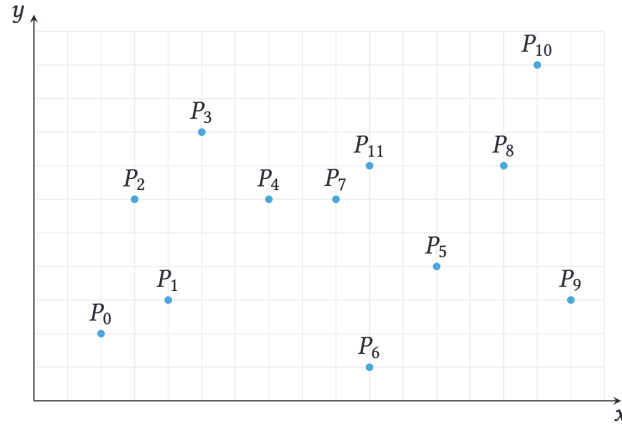
La distance entre deux points P_i et P_j de coordonnées respectives (x_i, y_i) et (x_j, y_j) est la distance euclidienne

$$d(P_i, P_j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

Le problème des *deux plus proches points du plan* consiste à identifier un couple de plus proches points au sens de cette distance euclidienne.

Une solution serait de calculer les distances entre tous les couples de points possibles, soit $n(n-1)/2$ distances, puis de trouver la plus petite d'entre elles.

Toutefois, cette solution de complexité quadratique peut être avantageusement remplacée par une approche *diviser pour régner* dont nous allons montrer qu'elle est en $\Theta(n \log n)$.



Désignons par a le tableau contenant l'ensemble des n couples de points. On commence par construire deux tableaux a_x et a_y qui contiennent les éléments de a triés respectivement par abscisses croissantes et par ordonnées croissantes.

- Si $n \leq 3$, un algorithme direct détermine les deux points les plus proches parmi les deux ou trois points, ce qui constitue un cas de base pour la suite de l'algorithme.
- Si $n \geq 4$, l'algorithme procède comme suit.
 - **Diviser.** Le tableau a_x est divisé en deux sous-tableaux $a_{x,g}$ et $a_{x,d}$ de tailles $\lfloor n/2 \rfloor$ et $\lceil n/2 \rceil$ respectivement. Les points stockés dans ces sous-tableaux sont situés de part et d'autre de la droite verticale Δ d'abscisse x_{med} , l'abscisse du point $a_x[\lfloor n/2 \rfloor - 1]$.
 - **Régner.** L'algorithme, appelé récursivement sur $a_{x,g}$ et $a_{x,d}$, détermine les deux distances minimales d_g et d_d ainsi que les couples de points les plus proches associés. Seul le couple correspondant à $d = \min(d_g, d_d)$ est conservé.

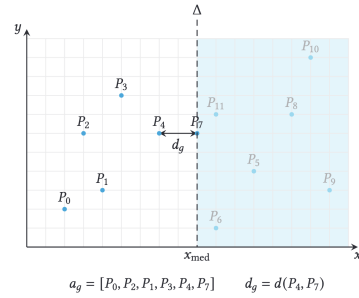


FIGURE 9.5 – Plus proches points dans a_g .

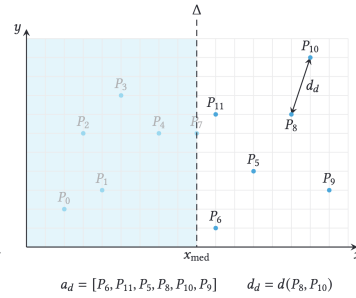


FIGURE 9.6 – Plus proches points dans a_d .

- **Rassembler.** Une fois un couple de plus proches points identifié dans a_g ou dans a_d , il convient d'étudier les points situés dans une bande verticale de largeur $2d$ centrée sur Δ . En effet, il est possible que deux points de cette bande soient plus proches que ceux déjà trouvés. En pratique, cette recherche se fait en considérant le sous-tableau de a_y qui ne contient que les points dont les abscisses sont comprises entre $x_{\text{med}} - d$ et $x_{\text{med}} + d$.

En y regardant de plus près, on remarque que, pour chaque point de la bande, il suffit de considérer uniquement les sept points qui le suivent dans le tableau.

Sans perte de généralité, supposons qu'un point P_i soit sur le côté inférieur du rectangle. Puisque tous les points de a_g sont distants d'au moins d , au plus quatre points sont dans le carré de largeur d et de hauteur d formant la moitié gauche du rectangle.

De même, au plus quatre points sont dans le carré de mêmes dimensions formant la moitié droite du rectangle.

Au total, huit points sont susceptibles de se trouver dans le rectangle. Si P_i est l'un de ces points, il ne reste que sept points avec lesquels il peut être à une distance inférieure à d .

Le code OCaml contenu dans les programmes suivantes met en œuvre cet algorithme.

Programme 1. Points. Distance. Comparaisons

```

1 type point = float * float
2
3 let dist ((x1,y1), (x2,y2)) =
4   sqrt ((x1-.x2) *. (x1-.x2) +. (y1-.y2) *. (y1-.y2))
5
6 let compare_x (x1, _) (x2, _) =
7   compare x1 x2
8
9 let compare_y (_, y1) (_, y2) =
10  compare y1 y2

```

Programme 2. Deux plus proches points du plan

```

1 let closest_pair_small (a: point array) : point * point =
2   match a with
3   | [| p; q |] ->
4     (p, q)
5   | [| p; q; r |] ->
6     let pq = dist (p, q) in
7     let qr = dist (q, r) in
8     let pr = dist (p, r) in
9     if pq < qr
10    then if pr < pq then (p, r) else (p, q)
11    else if pr < qr then (p, r) else (q, r)
12   | _ ->
13     assert false
14
15 let closest_pair_in_strip (a: point array) : point * point =
16   let n = Array.length a in
17   let d = ref (dist (a.(0), a.(1))) in
18   let p = ref 0 in
19   let q = ref 1 in
20   for i = 0 to n - 1 do
21     for j = i + 1 to min (n - 1) (i + 7) do
22       let e = dist (a.(i), a.(j)) in
23       if e < !d then (d := e; p := i; q := j)
24     done
25   done;
26   (a.(!p), a.(!q))
27
28 let select (a: point array) (xmin: float) (xmax: float)
29   : point array =

```

```

30 let lst = ref [] in
31 for i = Array.length a - 1 downto 0 do
32   let x, _ = a.(i) in
33   if xmin <= x && x <= xmax then lst := a.(i) :: !lst
34 done;
35 Array.of_list !lst
36
37 let closest_pair (a: point array) : point * point =
38   if Array.length a < 2 then invalid_arg "closest_pair";
39   let ax = Array.copy a in Array.sort compare_x ax;
40   let ay = Array.copy a in Array.sort compare_y ay;
41   let rec closest_rec lo hi =
42     if hi - lo <= 3 then
43       closest_pair_small (Array.sub ax lo (hi - lo))
44     else (
45       let mid = lo + (hi - lo) / 2 in
46       let pql = closest_rec lo mid in
47       let pqr = closest_rec mid hi in
48       let pq = if dist pql < dist pqr then pql else pqr in
49       let d = dist pq in
50       let xmed = fst ax.(mid - 1) in
51       let strip =
52         select ay (max (xmed -. d) (fst ax.(lo)))
53           (min (xmed +. d) (fst ax.(hi - 1))) in
54       if Array.length strip <= 1 then
55         pq
56       else (
57         let pqs = closest_pair_in_strip strip in
58         if dist pqs < d then pqs else pq
59       )
60     )
61   in
62   closest_rec 0 (Array.length a)

```

Les points sont ici des couples de flottants (type `point`).

La fonction principale, `closest_pair`, commence par faire deux copies du tableau `a` pour les trier respectivement selon les abscisses et les ordonnées, avec les fonctions de comparaison `compare_x` et `compare_y`.

Le principe diviser pour régner est réalisé par la fonction récursive `closest_rec` : elle reçoit en arguments deux indices `lo` et `hi` qui délimitent la portion du tableau à considérer.

Lorsque celle-ci ne contient que deux ou trois points, on appelle la fonction `closest_pair_small`. Sinon, le segment est divisé en deux moitiés, sur lesquelles on procède récursivement.

La bande est construite avec la fonction `select` puis examinée avec la fonction `closest_pair_in_strip` si elle contient au moins deux points.

Complexité. Pour un tableau contenant n couples de points, les deux tris de a ont un coût en $\Theta(n \log n)$.

Les appels récursifs à la fonction `closest_rec` ont un coût qui se décompose schématiquement

en deux termes :

- un terme de la forme $C(\lfloor n/2 \rfloor)$ correspondant aux deux appels récursifs sur chacun des sous-tableaux $a_{x,g}$ et $a_{x,d}$ de tailles voisines de $\lfloor n/2 \rfloor$;
- un terme de la forme $\Theta(n)$ correspondant à la division de a_x en $a_{x,g}$ et $a_{x,d}$, à la construction de la bande (appel à `select`) et à la recherche du couple de plus proches points dans la bande (appel à `closest_pair_in_strip`).

Ainsi, on obtient la relation de récurrence :

$$C(n) = C(\lfloor n/2 \rfloor) + \Theta(n)$$

ce qui mène à :

$$C(n) = \Theta(n \log n).$$